

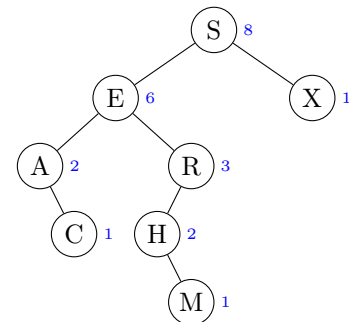
Lecture Review: Binary Search Trees (BSTs)

Definition. A *binary search tree* is a binary tree in *symmetric order*: every node has a key that is strictly larger than all keys in its left subtree and strictly smaller than all keys in its right subtree (duplicate keys are not permitted). The *depth* of a node is the number of links on the path from the root to it (the root has depth 0), and the *height* of a tree is the maximum depth of any of its nodes.

Search. Start at the root: if the key is less, go left; if greater, go right; if equal, search hit. Falling off the tree (reaching **nullptr**) is a search miss.

Insert. Search for the key. If it is found, reset its value; otherwise attach a new node at the **nullptr** link where the search ended.

Cost. Both take $1 + \text{depth of the deepest node reached compares}$, i.e., $O(h)$ for a tree of height h .



A BST; each node is labelled with the size of its subtree.

Tree shape. Many BSTs hold the same set of keys; the shape depends only on the order of insertion. The height lies between $\lfloor \log_2 n \rfloor$ (perfectly balanced) and $n - 1$ (keys inserted in sorted order). If the n keys are inserted in uniformly random order, the expected height is $\sim 4.311 \ln n$ and a search costs $\sim 2 \ln n$ compares on average.

Ordered operations. Symmetric order lets a BST answer every ordered symbol-table query by following a single root-to-leaf path:

- **min()** / **max()**: follow left / right links until the next one is **nullptr**.
- **floor(key)** / **ceiling(key)**: search for **key**. Every node on the search path whose key is $\leq \text{key}$ (resp. $\geq$) is a candidate, and the answer is the last candidate seen. For example, **floor(G)** = E and **ceiling(G)** = H in the tree above.
- **rank(key)** and **select(r)**: each node stores the *size* of its subtree (1+ size of left + size of right). At node **x**, **rank** adds $1 + \text{size}(x \rightarrow \text{left})$ whenever it goes right; **select** compares r with $t = \text{size}(x \rightarrow \text{left})$ and goes left ($r < t$), returns **x** ($r = t$), or goes right looking for rank $r - t - 1$. For example, **rank(M)** = 4 and **select(2)** = E.

Implementation	search	insert	ordered ops
Binary search (ordered array)	$\Theta(\log n)$	$\Theta(n)$	$\Theta(\log n)$
BST (worst case)	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
BST (random insertion order, average)	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$

Most BST methods are naturally *recursive*: a tree is either empty (**nullptr**) or a node together with two smaller trees. The **size** method at the end of this lab is the pattern to follow — handle the empty tree, then combine the answers from the two subtrees.

Lab Exercises

In this lab, you will implement various methods on binary search trees (BSTs). Assume the BST is defined using the following structure:

```

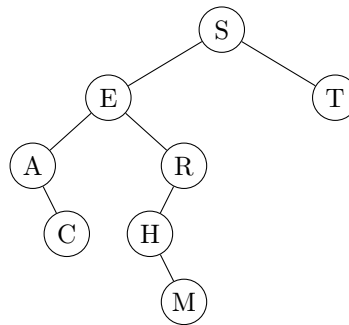
1 struct Node {
2     int key;
3     Node* left;
4     Node* right;
5     Node(int k) : key(k), left(nullptr), right(nullptr) {}
6 };

```

Part A: Level order traversal of a binary tree

Given a binary tree, return the level order traversal of its nodes' keys (i.e., from left to right, level by level). For the tree below, the level order traversal is S E T A R C H M.

Hint: Use a queue. Start with the root in the queue; repeatedly remove a node, record its key, and add its non-null children (left, then right) to the queue.



level order traversal: S E T A R C H M

Part B: Constructing BST from level order sequence

Given the level order traversal of a binary search tree, reconstruct the original tree and return its root. (For example, S E T A R C H M should give back the tree above.)

Hint: You can use either of the following approaches.

1. Insert the keys into an empty BST in the order given by the sequence. Every key is inserted after its parent, so this reproduces the original tree. The time complexity is $O(n \times h)$, where n is the number of keys and h is the height of the tree. In the worst case h is $\Theta(n)$, giving $O(n^2)$ time.
2. Use a queue of nodes, each paired with the range (min, max) of keys allowed in its subtree; the root's range is $(-\infty, \infty)$. Repeatedly remove a node x from the queue and look at the next unprocessed key k in the sequence:
 - if $min < k < x.key$, make k the left child of x , push it with range $(min, x.key)$, and move on to the next key;
 - then, if the next key k satisfies $x.key < k < max$, make it the right child of x and push it with range $(x.key, max)$.

Every key is placed directly in its final position without searching from the root, so this takes $O(n)$ time.

Part C: Recursive methods on binary trees

Given a binary search tree (BST), implement the following recursive methods. Each method takes the root of the tree as its first argument. Use the conventions from the lecture review: the root has depth 0, and the height of an empty tree is -1 .

1. `int height(Node* root)` – maximum depth of any node in the tree.
2. `int sizeOdd(Node* root)` – number of nodes with an odd key.
3. `bool isPerfectlyBalanced(Node* root)` – at every node, do the left and right subtrees have the same height?
4. `bool isSemiBalanced(Node* root)` – is every node semi-balanced? A node is *semi-balanced* if $\text{size}(\text{larger child}) + 1 \leq 2 \times (\text{size}(\text{smaller child}) + 1)$, where a missing child has size 0. (So a leaf, or a node whose only child is a leaf, is semi-balanced.)
5. `int sizeAtDepth(Node* root, int d)` – number of nodes at depth `d`.
6. `int sizeAboveDepth(Node* root, int d)` – number of nodes whose depth is $< d$.
7. `int sizeBelowDepth(Node* root, int d)` – number of nodes whose depth is $> d$.

The support files contain `bst.hpp`, with a stub for every method in this lab, and `test-bst.cpp`, a test driver whose comments give the expected output. Compile with `g++ -std=c++17 test-bst.cpp`.

To get you started, here is an example of `size` method that counts the number of nodes in the tree:

```
1 int size(Node* root) {  
2     if (root == nullptr) {  
3         return 0;  
4     }  
5     return 1 + size(root->left) + size(root->right);  
6 }
```